AURORA

Communications Protocol

Engineering Handoff Specification

ACP v1.2 • Swift + Python + Rust Reference Implementations

MISSION Define one stable, versioned, language-neutral contract that lets Prism, Conductor, Lyric, Bridge, tools, test harnesses, and future Aurora products communicate without product-specific socket code leaking into application logic.

Document	Value
Status	Implementation handoff / protocol baseline
Protocol name	Aurora Communications Protocol (ACP)
Baseline version	1.2
Required SDKs	Swift, Python, Rust
Primary environment	Trusted production LANs used for live shows
Design priority	Deterministic control, explicit state, graceful degradation, debuggability
Family scope	Prism, Conductor, Lyric, Bridge, future Aurora nodes

The protocol is infrastructure. It must stay boring when the show gets exciting.

1. Executive Summary

ACP is the common communications substrate for the Aurora family. It is not a replacement for Art-Net, sACN, DMX, MIDI, OSC, mixer-native APIs, or device drivers. Those protocols remain at the edges. ACP carries Aurora-level intent, state, health, configuration, synchronization, and control between Aurora products.

The protocol must allow Conductor to orchestrate a show without becoming a single giant code dependency, allow Prism to expose lighting state and accept authorized commands, allow Lyric clients to follow songs and sections, and allow Bridge nodes to report health, receive configuration, and perform lightweight edge functions.

NON-NEGOTIABLE Never infer that a command was applied merely because it was transmitted. ACP separates delivery, acceptance, application, and observed/confirmed state.

1.1 Core decisions

- Language-neutral schema with generated or hand-maintained strongly typed models.
- CBOR is the canonical compact wire encoding. JSON is a required human-readable/debug encoding with identical field names and semantics.
- Reliable sessions use WebSocket as the baseline transport because it works cleanly in Swift, Python, Rust, browser tooling, and local web interfaces. Raw TCP may be added later without changing message semantics.
- UDP multicast is used only for discovery advertisements and optional low-cost presence beacons. Safety-critical control never depends on multicast delivery.
- Every message has a common envelope containing protocol version, message ID, type, source, destination, session, sequence, timestamp, correlation, flags, and payload.
- Capabilities are negotiated at session establishment. Unknown optional capabilities must not break interoperability.
- Commands use explicit acknowledgements and state-confidence semantics. 'Sent' is not 'confirmed'.
- High-frequency telemetry is separated from durable state and may be rate-limited, coalesced, or dropped according to declared QoS.
- The protocol supports a Lightweight Profile for constrained Bridge targets such as Pi Pico W-class devices.
- ACP v1 must be testable without any Aurora GUI. A CLI test peer and conformance vectors are part of the deliverable.

1.2 Product roles

Product / Role	ACP responsibility
Conductor	Show-level orchestrator. Discovers nodes, establishes sessions, drives song/section/show progression, receives health, reconciles state, and exposes operator warnings.
Prism	Lighting controller. Publishes lighting/control capability, current show context and health; accepts authorized show/control actions; continues local operation during nonfatal ACP failures.
Lyric	Lyrics/charts client or standalone session participant. Receives show/song/section position, chart assignments and synchronization events; can participate in session-master workflows.
Bridge	Edge node. Hosts Art-Net/DMX and related I/O functions, heartbeat/health telemetry, local status web UI, configuration, blackout, and optional lightweight profile.
Tools / Simulators	Protocol inspectors, discovery tools, replay harnesses, fixture simulators, test peers, and diagnostics.

2. Scope and Non-Scope

2.1 In scope

- Node identity, discovery, connection, authentication hooks, capability negotiation, session lifecycle.
- Request/response commands, events, state snapshots, state deltas, telemetry, logs, warnings, errors.
- Show, song, section, cue, transport, synchronization, and operator-control semantics.
- Bridge configuration and status, including safe blackout control.
- Health and heartbeat semantics with explicit severity and confidence.
- Version negotiation, compatibility rules, extension namespaces, and feature flags.

- Reference SDK architecture for Swift, Python, and Rust.
- Conformance tests, golden CBOR/JSON vectors, fuzz/property testing targets, and simulated peers.

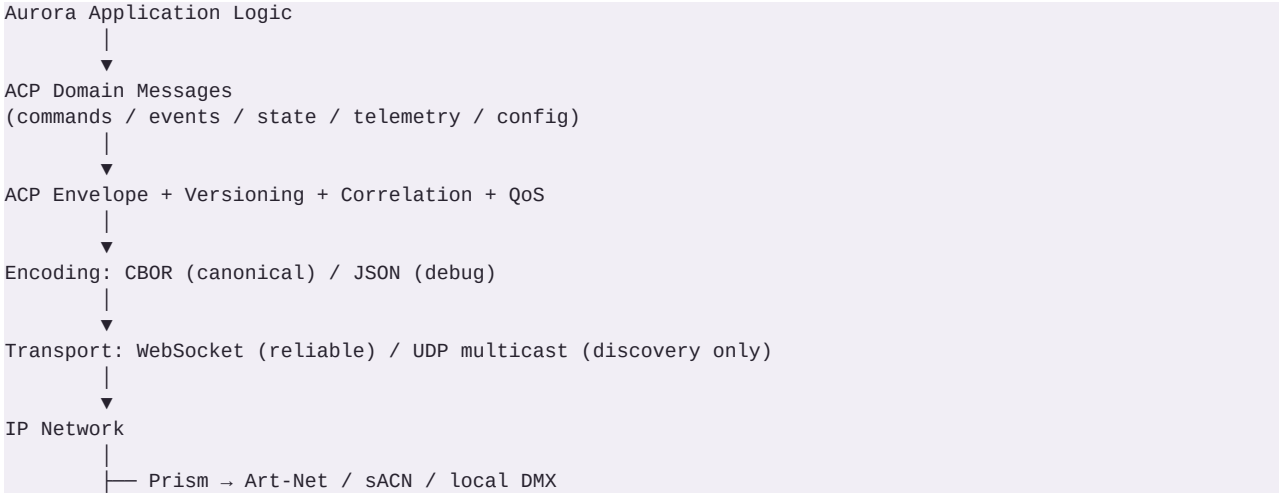
2.2 Explicitly out of scope

- Replacing Art-Net, sACN, DMX512, MIDI, RTP-MIDI, OSC, Behringer Wing protocols, or vendor APIs.
- Streaming raw DMX universes between all Aurora products as the normal control model. Bridge may expose an edge-specific stream capability, but Aurora semantic control remains preferred.
- Audio/video media transport. ACP may carry cues, transport state, metadata, and synchronization markers, not bulk media.
- A cloud service requirement. ACP v1 must work entirely on a private LAN with no Internet dependency.
- A mandatory licensing or phone-home mechanism.
- Renaming existing internal Prism/Aurora datatypes solely for branding. Protocol models are a new boundary and may use ACP-specific names.

3. Architectural Principles

1. Semantic intent over device packets. Conductor tells Prism what Aurora action should occur; Prism remains responsible for translating that into its own engine behavior.
2. One authoritative owner per state kind. ACP transports ownership; it does not create competing owners.
3. Preview and physical output remain downstream of Prism's same resolved semantic state. ACP must never create a parallel lighting engine.
4. GO must remain usable during nonfatal network or peripheral failures. ACP failures should degrade capability, not freeze the show.
5. Control plane and telemetry plane are logically distinct, even when sharing a WebSocket.
6. Idempotency is designed in. Commands that can be safely retried declare an idempotency key.
7. State can be reconstructed. New/reconnected peers can request snapshots instead of relying on a perfect event history.
8. Every protocol feature must be inspectable. A human-readable JSON representation is mandatory.
9. Unknown fields are ignored unless a message explicitly marks them required.
10. Safety beats cleverness. Ambiguous commands fail closed; blackout and emergency-like actions are explicit, scoped, logged, and acknowledged.

4. Layer Model



```
        ├── Conductor → MIDI / vendor drivers / show control
        ├── Bridge → Art-Net / DMX / GPIO / local web
        └── Lyric → local presentation UI
```

5. Identity and Addressing

Every ACP participant is a node. A node has a stable node UUID, a human-readable name, a product role, an instance ID for the current process boot, and optional endpoint/component IDs.

Field	Type	Meaning
node_id	UUID	Stable installation identity. Persisted across restarts.
instance_id	UUID	New UUID for each process/device boot. Detects restarts.
role	enum	conductor | prism | lyric | bridge | tool | simulator | future extension
name	string	Operator-facing label, e.g. 'Stage Left Bridge'.
component_id	string?	Optional subcomponent address, e.g. prism.output.artnet.
session_id	UUID?	ACP logical session established after handshake.

Destination may be a specific node UUID, a role-scoped logical target, or broadcast within a session. Broadcast commands that can change show state are discouraged; orchestration should target known participants.

6. Discovery

ACP discovery exists so Conductor and diagnostic tools can find Aurora products on a show LAN without manual IP entry. Discovery must not be treated as authentication.

6.1 Multicast baseline

- Use a dedicated Aurora UDP multicast group and port selected from administratively scoped ranges during implementation. Keep both constants centralized in the protocol package.
- Discovery packets are CBOR by default and must remain under a single safe UDP datagram size. No fragmentation.
- A node sends an advertisement at startup, after material network changes, and periodically with jitter.
- A controller may send DISCOVERY_QUERY; nodes answer with DISCOVERY_ADVERTISEMENT after randomized short jitter to avoid response storms.
- Advertisement contains node identity, role, product version, ACP min/max version, reliable endpoint URL, capabilities digest, and security mode.

IMPLEMENTATION NOTE Do not hard-code a guessed multicast address in product code before the protocol package owns the constant. The SDK must expose the chosen address/port as protocol constants.

7. Reliable Session Transport

WebSocket is the ACP v1 reliable transport. Each node that accepts inbound ACP sessions exposes a WebSocket endpoint. Conductor may also accept inbound sessions so constrained nodes can operate in client-only mode.

- Binary WebSocket frames carry CBOR. Text frames carry JSON and are intended for diagnostics, development, browser tooling, and optional local web interfaces.

- One ACP envelope per WebSocket message in v1. Do not concatenate envelopes.
- Ping/pong at the WebSocket layer is transport liveness only. ACP heartbeat messages carry application health.
- Reconnect uses exponential backoff with jitter and an upper bound appropriate for live-show recovery.
- After reconnect, perform a fresh HELLO/HELLO_ACK handshake and request/emit state snapshots as needed. Do not assume the prior session survived.

8. Common Envelope

All reliable ACP messages use the same envelope. JSON names below are normative. CBOR uses the same logical keys; implementations may later assign compact integer keys only in a new negotiated encoding revision.

```
{
  "acp": "1.0",
  "message_id": "0193f8d8-4c4e-7d8b-a2ab-...",
  "type": "command.execute",
  "source": {"node_id":"...", "component_id":"conductor.show"},
  "destination": {"node_id":"...", "component_id":"prism.control"},
  "session_id": "...",
  "sequence": 1842,
  "timestamp_utc": "2026-08-17T16:42:15.231Z",
  "correlation_id": "...",
  "causation_id": "...",
  "qos": "reliable",
  "flags": ["ack_required"],
  "payload": { ... }
}
```

Envelope field	Required	Rule
acp	yes	Major.minor protocol version used to encode semantics.
message_id	yes	Globally unique UUID; UUIDv7 preferred where available.
type	yes	Namespaced message type.
source	yes	Node and optional component.
destination	no	Absent for session-wide event/telemetry where allowed.
session_id	reliable	Required after HELLO completes.
sequence	reliable	Monotonic per session and sender, uint64.
timestamp_utc	yes	RFC3339/ISO-8601 UTC wall-clock timestamp.
correlation_id	no	Links response/ack to request.
causation_id	no	Links derived event/state change to the triggering message.
qos	yes	reliable | latest | best_effort.
flags	yes	Array, empty allowed. Unknown optional flags ignored.
payload	yes	Message-specific object/map.

9. Time, Ordering, and Synchronization

- Wall-clock UTC is for logs, correlation, and cross-node observability. It is not sufficient for precise local scheduling.
- Each session maintains sequence numbers to detect gaps and duplicates.

- Scheduled actions may include execute_at_utc plus tolerance_ms. A receiver must reject or immediately execute according to explicit late_policy.
- For music/show progression, prefer semantic markers such as song position, section entry, beat/bar index, or cue ID over trying to synchronize via packet arrival time.
- ACP v1 may report clock quality and offset estimates, but it is not a replacement for PTP/NTP. Future high-precision synchronization can add a negotiated clock capability.

10. Message Families

Prefix	Purpose	Examples
session.*	Handshake and lifecycle	session.hello, session.hello_ack, session.goodbye
discovery.*	LAN discovery	discovery.query, discovery.advertisement
command.*	Requested action	command.execute, command.ack
state.*	Authoritative state	state.snapshot, state.delta, state.request
health.*	Node/application health	health.heartbeat, health.warning, health.snapshot
show.*	Show-level context	show.load, show.ready, show.position
song.*	Song context	song.select, song.start, song.stop, song.position
section.*	Musical section progression	section.enter, section.exit, section.position
cue.*	Cue-level control	cue.go, cue.fire, cue.stop
transport.*	Generic transport	transport.play, pause, stop, seek
config.*	Configuration	config.schema, config.get, config.set, config.result
capability.*	Feature discovery	capability.list, capability.changed
telemetry.*	High-rate observability	telemetry.metric, telemetry.batch
log.*	Structured diagnostics	log.event
error.*	Protocol/application errors	error.report

11. Handshake and Capability Negotiation

```
Client → session.hello
{
  "node": {
    "node_id":"...", "instance_id":"...", "role":"bridge",
    "name":"Stage Right Bridge", "product_version":"0.9.0"
  },
  "protocol": {"min":"1.0","max":"1.0"},
  "encodings":["cbor","json"],
  "profiles":["core","bridge","lightweight"],
  "capabilities":[
    {"id":"bridge.artnet_output","version":"1.0"},
    {"id":"bridge.blackout","version":"1.0"},
    {"id":"health.heartbeat","version":"1.0"}
  ],
  "auth": {"mode":"trusted_lan"}
}

Server → session.hello_ack
{
  "accepted": true,
  "protocol":"1.0",
  "encoding":"cbor",
  "session_id":"...",
  "heartbeat_interval_ms":1000,
```

```
  "peer_capabilities":[ ... ],
  "limits":{"max_message_bytes":1048576}
}
```

Capability IDs are stable strings. Capability versions evolve independently from ACP minor versions where practical. A peer must not send capability-specific commands until the capability is negotiated.

12. Commands, Acknowledgements, and Confirmation

ACP distinguishes transport delivery from semantic result. The receiver must produce command.ack when ack_required is set.

Status	Meaning
accepted	Command is syntactically valid and accepted for processing.
rejected	Command will not be processed. Error details required.
applied	Receiver has applied the command to its authoritative local state.
completed	Long-running operation finished successfully.
failed	Processing began but failed.
duplicate	Idempotency key/message was already processed; prior result may be returned.

For external devices, application state must also expose confidence. This is essential for MIDI and other interfaces without positive acknowledgement.

Confidence	Meaning
unverified	No claim that external state matches desired state.
sent_assumed	Command was transmitted; no feedback path exists.
pending	Feedback/query is expected but not yet resolved.
confirmed	Device feedback or authoritative query confirms desired state.
mismatch	Observed state differs from desired state.

RULE Never infer CONFIRMED from a successful socket write, MIDI send, or driver transmit call.

13. State Model

State is modeled as named resources with an owner, revision, value, and confidence where applicable. Receivers can request a full snapshot after connection or a detected sequence gap.

```
{
  "type":"state.delta",
  "payload":{
    "resource":"show.position",
    "revision":418,
    "owner":{"node_id":"<conductor>"},
    "value":{
      "show_id":"...",
      "song_id":"...",
      "section_id":"chorus_2",
      "bar":57,
      "beat":1
    },
    "confidence":"confirmed"
  }
}
```

- revision is monotonically increasing per resource owner.
- state.delta may be coalesced. state.snapshot is complete for the declared resource set.
- A non-owner may request change via command but must not publish itself as authoritative owner.

- Ephemeral UI state should not automatically become ACP state. Only cross-product state belongs here.

14. Health and Heartbeats

Heartbeat support is first-class because Bridge and Conductor depend on node health visibility. Heartbeats report application health, not merely socket liveness.

```
{
  "type":"health.heartbeat",
  "qos":"latest",
  "payload":{
    "uptime_ms":8294412,
    "status":"degraded",
    "summary":"DMX output active; MIDI input unavailable",
    "metrics":{
      "cpu_pct":22.4,
      "memory_pct":41.0,
      "temperature_c":54.1
    },
    "components":[
      {"id":"artnet","status":"ok"},
      {"id":"dmx","status":"ok"},
      {"id":"midi","status":"warning","code":"device_disconnected"}
    ]
  }
}
```

Health status	Operator meaning
ok	Operating normally.
degraded	Core function continues; one or more capabilities impaired.
warning	Attention recommended; show can generally continue.
critical	Primary function at risk or unavailable.
offline	Derived by observer after heartbeat/session timeout, not normally self-reported.

- Heartbeat interval is negotiated. 1 second is a sensible full-profile default; lightweight nodes may negotiate slower intervals.
- Observers use missed-heartbeat thresholds with hysteresis. Do not flash online/offline on one lost packet.
- Warnings have stable codes, severity, first_seen, last_seen, count, and optional remediation text.
- Health must distinguish engine/application health from transport capability. A connected Art-Net socket does not prove fixtures are responding.

15. Show and Music Semantics

Aurora is music-centered. ACP therefore carries explicit show, song, and musical-section context rather than reducing everything to generic remote buttons.

Message	Purpose
show.load	Request a product load or prepare the identified show/project.
show.ready	Report readiness and any degraded dependencies.
song.select	Select/preload a song without necessarily starting it.
song.start	Enter live playback/performance context.
song.position	Publish semantic position: elapsed, bar/beat, section, optional tempo.
section.enter	Musician-, operator-, MIDI-, or automation-driven transition into a named section.
cue.go	Advance the receiving product's current cue context using its own authoritative engine.
cue.fire	Fire a specific stable cue ID.

Musician-driven progression, Stream Deck/keyboard actions, MIDI events, and other input sources should converge in Conductor/Prism on shared behavior/action abstractions before ACP emission. ACP should not encode UI gestures such as 'button 7 clicked' when a semantic action exists.

16. Prism Profile

- Capabilities should include project/show identity, cue control, song context, busking/live-state controls where explicitly exposed, output health, and preview/status telemetry.
- ACP commands must enter Prism through the same ControlActionRouter/behavior path used by UI, MIDI, OSC, and remote control. No direct ACP-to-DMX shortcut.
- Prism remains authoritative for lighting engine state and output resolution.
- Remote GO, cue fire, blackout, programmer/live transitions, and other destructive/live actions must be permission-gated by capability and session policy.
- Prism should publish quiet health/status for Engine, DMX, MIDI, Art-Net/sACN, and other configured outputs without flooding the UI.

17. Conductor Profile

- Conductor is normally the show-context authority and may be the preferred ACP session coordinator, but ACP must not require every topology to have Conductor present.
- Conductor receives Bridge heartbeats, Prism state, Lyric readiness, and driver/device confidence and presents actionable warnings.
- Conductor must model external device state using unverified / sent_assumed / pending / confirmed / mismatch semantics.
- Conductor can issue semantic progression commands driven by musicians, keyboard shortcuts, Stream Deck, MIDI, automation, or operator controls.
- Redundant Conductor designs must not allow two active authorities to issue conflicting show progression. ACP exposes authority_epoch/lease hooks for future active/standby arbitration.

18. Lyric Profile

- Lyric receives show/song/section context, chart assignment metadata, position, and presentation directives.
- A Lyric client reports readiness, currently loaded chart/revision, display mode, and errors such as missing content.
- ACP supports both Conductor-client operation and a standalone Lyric session with a designated session master.
- Chart payload transfer should use a resource-transfer capability rather than stuffing large documents into ordinary control messages. v1 may define metadata and URI/hash references first.
- Per-client chart type preference (e.g. chord+lyrics vs Nashville Number) is client state/capability, not a global assumption.

19. Bridge Profile

Bridge is the primary edge-node profile and is where ACP's constrained-device design matters most.

Capability	Minimum semantics
bridge.status	Network, uptime, output mode, universe/port status, last error.
bridge.blackout	Explicit scoped blackout command with ack and resulting

	state.
bridge.config	Read/write validated configuration resources.
bridge.artnet_output	Art-Net node health/config metadata; actual Art-Net remains edge protocol.
bridge.dmx_output	DMX port state, rate, errors, output enabled.
health.heartbeat	Periodic application heartbeat.
log.event	Rate-limited structured diagnostics.

Bridge configuration should map naturally to the preferred TOML configuration file, but ACP transmits typed configuration values, not raw TOML text by default. A config export/import capability may carry full TOML as an explicit file/resource operation.

BLACKOUT SAFETY bridge.blackout must be idempotent, explicitly scoped, acknowledged, logged, and reflected in authoritative state. A reconnect must not silently clear blackout unless the configured safety policy says so.

20. Configuration Protocol

```
config.get
{
  "path":"bridge.outputs.dmx.0"
}

config.set
{
  "transaction_id":"...",
  "expected_revision":12,
  "changes":[
    {"path":"bridge.outputs.dmx.0.universe","value":1},
    {"path":"bridge.outputs.dmx.0.enabled","value":true}
  ],
  "apply":"validate_then_commit"
}

config.result
{
  "transaction_id":"...",
  "status":"applied",
  "new_revision":13,
  "restart_required":false,
  "field_results":[ ... ]
}
```

- Use optimistic concurrency via expected_revision to prevent stale Conductor edits from overwriting local changes.
- Validation errors are field-addressable and machine-readable.
- Changes that require restart must say so before commit when possible.
- Secrets are never returned by config.get. Secret fields are write-only or represented as 'configured': true.
- Configuration schema is discoverable so Conductor can build forms without hard-coding every Bridge firmware version.

21. QoS and Backpressure

QoS	Behavior	Use
reliable	Must be delivered in order or session fails/reconnects.	Commands, config, state snapshots, errors.
latest	Only newest value matters; older queued	Meters, heartbeat metrics,

	values may be replaced.	playhead/position.
best_effort	May be dropped under pressure.	Verbose diagnostics, optional traces.

- SDKs must have bounded outbound queues.
- Reliable queue overflow is a fault and must surface health/error; it must not silently discard commands.
- Latest-value channels should coalesce by resource key.
- Telemetry publishers declare nominal and maximum rates.
- High-frequency data must not force whole-application UI invalidation; product adapters should throttle immutable snapshots for UI consumption.

22. Error Model

```
{
  "type":"error.report",
  "payload":{
    "code":"config.revision_conflict",
    "category":"conflict",
    "severity":"warning",
    "message":"Configuration changed since revision 12.",
    "retryable":true,
    "details":{"expected":12,"actual":13}
  }
}
```

Category	Examples
protocol	Malformed envelope, unsupported version, invalid message type.
validation	Missing field, invalid range, incompatible configuration.
authorization	Capability not permitted for this session.
not_found	Unknown cue/song/resource.
conflict	Revision conflict, authority conflict.
unavailable	Subsystem offline or dependency missing.
timeout	Operation did not complete in required window.
internal	Unexpected implementation failure.

23. Security Model

ACP v1 targets controlled show LANs, but the protocol must not paint itself into an insecure corner.

- Security mode is negotiated and visible: trusted_lan for development/closed networks; tls and mutual_tls hooks reserved for hardened deployments.
- Authentication and authorization are transport/session concerns. Message semantics do not change when TLS is enabled.
- Capabilities are permission-filtered. A peer should only see or invoke operations allowed by policy.
- No remote code execution, shell command, arbitrary filesystem path, or generic 'run script' primitive belongs in ACP core.
- Configuration secrets are redacted. Logs must avoid credentials and sensitive tokens.
- Discovery advertisements expose only what is needed to connect and identify a node.

24. Lightweight Profile

The Lightweight Profile exists for Bridge builds on smaller devices such as Pi Pico W-class hardware. It is a subset of ACP, not a separate protocol.

- CBOR only is permitted; JSON support may be omitted.

- Client-only reliable connection is permitted so the device does not need a full inbound WebSocket server.
- Smaller maximum message size and fixed/bounded buffers are negotiated.
- Capabilities may be compiled out entirely.
- No dynamic schema reflection is required; config schema may be a compact fixed descriptor.
- Telemetry is aggressively rate-limited and batched.
- The device must still support identity, HELLO, heartbeat, command acknowledgement, errors, and its declared Bridge capabilities.
- Rust implementation should make alloc/std requirements explicit and isolate a core model layer that can migrate toward no_std where practical.

25. Versioning and Compatibility

- ACP uses MAJOR.MINOR. Major changes may break wire/semantic compatibility. Minor changes are additive.
- A v1.x implementation must ignore unknown optional fields and unknown optional flags.
- Unknown message types produce unsupported_message only when directed to that peer; observers may ignore them.
- Required feature additions are capability-gated, not smuggled into old semantics.
- Each capability has its own version string where independent evolution is useful.
- Deprecated fields remain readable for at least one minor compatibility window before removal in a future major version.
- Golden vectors are versioned and never silently rewritten.

26. Schema Source of Truth

Grok should create a protocol repository/module whose schema directory is the language-neutral source of truth. The initial implementation may use JSON Schema files plus prose invariants, with CBOR using the same logical model.

```
acp/
  schema/
    envelope.schema.json
    session/
    command/
    state/
    health/
    show/
    song/
    section/
    cue/
    config/
    bridge/
  vectors/
    json/
    cbor/
  docs/
    ACP_SPEC.md
    CAPABILITIES.md
    ERROR_CODES.md
  swift/
  python/
  rust/
  tools/
    acp-inspect/
    acp-sim/
    wireshark/
      acp.lua
```

```
    acp-coloring-rules.txt
    README.md
    captures/
```

Do not generate language APIs blindly if the generated result is unidiomatic. Schema conformance is mandatory; language ergonomics are allowed.

27. Swift Reference Implementation

Target modern Swift suitable for Prism, Conductor, and Apple-platform Lyric. Use Codable-style value types and async/await. Networking should be behind protocols so URLSessionWebSocketTask can be replaced/tested.

```
public struct ACPEnvelope<Payload: Codable & Sendable>: Codable, Sendable {
    public let acp: ACPVersion
    public let messageID: UUID
    public let type: ACPMessageType
    public let source: ACPEndpoint
    public let destination: ACPEndpoint?
    public let sessionID: UUID?
    public let sequence: UInt64?
    public let timestampUTC: Date
    public let correlationID: UUID?
    public let causationID: UUID?
    public let qos: ACPQoS
    public let flags: Set<ACPFlag>
    public let payload: Payload
}
```

- Modules: ACPModel, ACPEncoding, ACPTransport, ACPDiscovery, ACPSession, ACPProfiles, ACPTesting.
- All public protocol models should be Sendable where valid.
- Session actor owns sequence numbers, handshake state, reconnect policy, outbound queues, and correlation waiters.
- No network callbacks mutate SwiftUI state directly. Product adapters publish throttled immutable snapshots.
- Provide AnyACPMessage/type-erasure or registry-based decoding for heterogeneous payloads.
- Unit tests must round-trip every message in JSON and CBOR and compare against golden vectors.

28. Python Reference Implementation

Python is required for simulators, test harnesses, conversion utilities, diagnostics, and rapid integration work.

```
@dataclass(frozen=True, slots=True)
class ACPEnvelope(Generic[T]):
    acp: ACPVersion
    message_id: UUID
    type: str
    source: ACPEndpoint
    destination: ACPEndpoint | None
    session_id: UUID | None
    sequence: int | None
    timestamp_utc: datetime
    correlation_id: UUID | None
    causation_id: UUID | None
    qos: ACPQoS
    flags: frozenset[str]
    payload: T
```

- Python 3.11+ baseline unless repository constraints require newer.
- Use asyncio for transport/session implementation.
- Prefer dataclasses plus explicit validation or a lightweight validation layer; do not make a heavyweight runtime dependency mandatory for the core model package.

- Provide acp-inspect and acp-sim command-line tools.
- Expose async iterators for events/telemetry and awaitable request(command) correlation.
- Golden vector and property tests must be shared conceptually with Swift/Rust.

29. Rust Reference Implementation

Rust is required for Bridge and other performance/constrained Aurora components. The design should work well on Linux/Raspberry Pi and leave a path toward embedded targets.

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Envelope<T> {
    pub acp: Version,
    pub message_id: Uuid,
    #[serde(rename = "type")]
    pub message_type: String,
    pub source: Endpoint,
    pub destination: Option<Endpoint>,
    pub session_id: Option<Uuid>,
    pub sequence: Option<u64>,
    pub timestamp_utc: DateTime<Utc>,
    pub correlation_id: Option<Uuid>,
    pub causation_id: Option<Uuid>,
    pub qos: Qos,
    pub flags: BTreeSet<String>,
    pub payload: T,
}
```

- Crates/modules: acp-model, acp-codec, acp-session, acp-discovery, acp-profiles, acp-testkit.
- Use serde-compatible models for full profile. Keep codec traits abstract enough to support compact embedded CBOR implementations.
- Tokio is appropriate for full Linux/Raspberry Pi builds, but acp-model must not depend on Tokio.
- Avoid unbounded channels. Use explicit bounded queues and backpressure behavior.
- Separate full-profile std networking from a lightweight core so embedded Bridge builds can trim features.
- Fuzz CBOR/JSON decoders and message registry dispatch. Malformed input must never panic the process.

30. Optional Language Implementations

No additional language is required for ACP v1. TypeScript is the only optional implementation worth planning immediately because Bridge hosts a local web interface and browser-based diagnostics may benefit from typed ACP JSON models. It should remain JSON-only initially and should not delay Swift/Python/Rust.

PRIORITY Ship Swift, Python, and Rust first. TypeScript is optional tooling, not a fourth blocking reference implementation.

31. Core API Shape

All three SDKs should expose conceptually equivalent layers even when language idioms differ.

Layer	Responsibility
Model	Envelope, IDs, enums, payload types, capability descriptors.
Codec	JSON/CBOR encode/decode, validation, size limits.
Transport	WebSocket send/receive and discovery UDP.
Session	Handshake, sequence, reconnect, heartbeat, correlation, queues.
Router	Message type registry and typed handler dispatch.

Profiles	Prism, Conductor, Lyric, Bridge capability/message helpers.
TestKit	Fake transport, deterministic clock, simulated peers, golden vectors.

```
session = connect(peer)
await session.handshake(local_identity, capabilities)

ack = await session.request(
    CueGo(target=prism_id),
    timeout=0.500
)

async for hb in session.subscribe("health.heartbeat"):
    health_model.update(hb)
```

32. Message Registry and Extensibility

Each SDK maintains a registry mapping message type strings to payload decoders. Core types are registered by default; profile packages add their own.

- Type names are lowercase dot-separated ASCII identifiers.
- Aurora-reserved namespace is the unprefixed core/profile set defined by this spec.
- Experimental/vendor extensions use x.<organization>.<name>.
- Never overload an existing type with incompatible payload meaning.
- Handlers should be able to receive an UnknownMessage wrapper containing the raw payload for forward-compatible diagnostics.

33. Resource Transfer Extension

Large artifacts such as Lyric charts, project bundles, firmware, or exported TOML should not ride in ordinary control envelopes. Define an optional resource.transfer capability.

- Metadata message: resource ID, media type, byte length, SHA-256, revision, purpose.
- Transfer may use chunked ACP binary messages for small/medium files or an authenticated local HTTP URL for larger files.
- Receiver verifies hash before activation.
- Transfer and activation are separate operations.
- ACP core must remain usable when resource.transfer is absent.

34. Authority, Redundancy, and Split-Brain Hooks

Aurora is expected to grow into redundant show-control topologies. v1 should include fields/hooks without attempting a full consensus protocol.

- Authoritative state resources may include authority_node_id and authority_epoch.
- A future lease capability can add lease_id and expires_at.
- Receivers should reject conflicting lower-epoch authority updates when the capability is active.
- Prism and Bridge must continue safe local behavior if Conductor disappears; reconnect does not imply an automatic state jump without reconciliation.

35. Wireshark Dissector and Display Filters

A first-class Wireshark integration is a required ACP developer tool. Troubleshooting live-show networks must not require manually decoding CBOR or guessing which WebSocket frame contains a heartbeat, cue transition, configuration write, or rejected command.

35.1 Deliverable

- Ship an Aurora ACP Wireshark dissector, initially as a Lua plugin for rapid deployment and iteration. A native C dissector may be added later if upstreaming or performance justifies it.
- The dissector must recognize ACP discovery UDP datagrams and ACP reliable-session traffic. For WebSocket sessions, it must decode ACP payloads after transport identification and expose protocol fields to Wireshark display filters.
- The dissector must understand both canonical CBOR frames and JSON diagnostic frames.
- The ACP repository must include installation instructions for macOS, Windows, and Linux Wireshark plugin directories, plus a one-command developer installation helper where practical.
- Provide sample PCAP/PCAPNG captures and expected decoded screenshots/text output as regression fixtures.

35.2 Required display-filter fields

Filter field	Meaning / example
acp	Any decoded ACP packet/frame.
acp.version	Protocol version, e.g. acp.version == "1.0".
acp.type	Message type, e.g. acp.type == "health.heartbeat".
acp.message_id	Unique message identifier.
acp.correlation_id	Request/ack correlation identifier.
acp.causation_id	Trigger chain identifier.
acp.source.node_id	Originating Aurora node UUID.
acp.source.component_id	Originating component.
acp.destination.node_id	Target Aurora node UUID.
acp.session_id	Reliable ACP session.
acp.sequence	Per-sender session sequence number.
acp.qos	reliable | latest | best_effort.
acp.command.status	accepted | rejected | applied | completed | failed | duplicate.
acp.state.resource	State resource path.
acp.state.revision	Authoritative resource revision.
acp.state.confidence	unverified | sent_assumed | pending | confirmed | mismatch.
acp.health.status	ok | degraded | warning | critical.
acp.error.code	Stable machine-readable ACP error code.
acp.capability.id	Negotiated capability identifier.
acp.bridge.blackout	Decoded Bridge blackout state/command where present.

35.3 Troubleshooting filters that must work

```
# All ACP traffic
acp

# Heartbeats from Bridge nodes
acp.type == "health.heartbeat" && acp.source.role == "bridge"
```

```
# Anything unhealthy
acp.health.status == "degraded" || acp.health.status == "warning" || acp.health.status == "critical"

# Command failures/rejections
acp.command.status == "rejected" || acp.command.status == "failed"

# Follow one request/response chain
acp.correlation_id == <UUID>

# Prism cue operations
acp.type == "cue.go" || acp.type == "cue.fire"

# Bridge blackout activity
acp.bridge.blackout

# Configuration changes and conflicts
acp.type == "config.set" || acp.error.code == "config.revision_conflict"

# Suspected external-device confirmation problems
acp.state.confidence == "mismatch" || acp.state.confidence == "sent_assumed"
```

35.4 Packet-detail tree

- Wireshark packet details should present an Aurora Communications Protocol root with Envelope, Identity/Addressing, Session, Payload, and Diagnostics subtrees.
- Known payload types should expose typed fields rather than only a raw CBOR map. Unknown fields remain visible as decoded generic key/value data.
- Malformed ACP should be marked with Wireshark expert information rather than causing the dissector to throw or stop decoding later traffic.
- Sequence gaps, duplicate message IDs, unsupported protocol versions, and malformed timestamps should emit expert-info warnings when detectable from the capture.
- Secrets and write-only configuration values must not be specially reconstructed or exposed beyond what actually appears on the wire.

35.5 Colorization recommendations

- Provide an optional Wireshark coloring-rules file: critical/errors in a strong warning color, degraded/warnings in amber-like emphasis, show/cue control in a distinct control color, heartbeats/telemetry subdued, and discovery/session setup visually separate.
- Coloring is developer convenience only; filtering and decoding must not depend on custom colors.

35.6 Dissector architecture

- Keep dissector field names stable because documentation, troubleshooting runbooks, and saved Wireshark profiles will depend on them.
- Generate or share message-type/field metadata from the ACP schema where practical so the dissector does not drift from Swift/Python/Rust implementations.
- The Lua dissector must be version-aware and safely display unknown future fields.
- If WebSocket dynamic-port detection is ambiguous, support Decode As and/or a configurable ACP port preference in addition to automatic detection.
- Discovery traffic should be directly recognizable from the dedicated ACP UDP discovery port and magic/version marker.

36. Observability and Logging

- Every command lifecycle can be traced by message_id, correlation_id, and causation_id.

- Structured logs use stable code + human message + fields.
- SDKs expose counters: messages sent/received, decode errors, reconnects, queue depth, dropped latest/best-effort messages, RTT, heartbeat age.
- Provide a protocol inspector capable of rendering CBOR as readable JSON.
- Support capture/replay of ACP envelopes with timestamps for smoke testing, but replay must never accidentally target a live show without explicit enablement.

37. Conformance and Test Requirements

Test class	Required checks
Golden vectors	Every normative message has canonical JSON and CBOR examples consumed by all SDKs.
Round-trip	Encode → decode preserves semantic equality.
Cross-language	Swift-generated vectors decode in Python/Rust and vice versa.
Malformed input	Bad lengths/types/UUIDs/timestamps/unknown fields fail safely.
Sequence	Duplicate, gap, reconnect, rollover assumptions.
Correlation	Timeout, late ack, duplicate ack, wrong correlation.
Backpressure	Bounded queues, coalescing, reliable overflow fault.
Discovery	Startup advert, query jitter, duplicate nodes, restart instance ID.
Heartbeat	Degraded/offline hysteresis and restart detection.
Config	Validation, revision conflict, atomic apply, restart-required.
Safety	Blackout idempotency, permission rejection, reconnect behavior.
Fuzz	Rust/Python decoder fuzz/property tests; Swift malformed corpus.

A release is not conformant merely because each SDK passes its own unit tests. At least one automated cross-language integration test must connect two different implementations over WebSocket and exercise HELLO, command/ack, heartbeat, state snapshot, and disconnect/reconnect.

38. Initial Normative Message Set for v1.0

Required Core	Required Profiles
session.hello / hello_ack / goodbye	Prism: cue.go, cue.fire, state/health
command.execute / command.ack	Conductor: show/song/section context
state.request / snapshot / delta	Lyric: ready, chart metadata, song/section position
health.heartbeat / warning / snapshot	Bridge: status, blackout, config
capability.list / changed	All: profile capability descriptors
error.report	All: profile-specific stable error codes

Grok may define convenience strongly typed command payloads instead of forcing all operations through a generic command.execute. The generic command form is useful for tooling, but normative typed messages are preferred for important operations.

39. Suggested Stable IDs and Data Types

Concept	Type
Node/session/message IDs	UUID
Show/song/cue IDs	Stable UUID or existing stable product identifier serialized as string
Section ID	Stable string/UUID scoped to song

Revision	uint64
Sequence	uint64
Durations	integer milliseconds unless higher precision explicitly required
Rates/percentages	finite numeric values with documented range
Timestamps	RFC3339 UTC in JSON; tagged/standard datetime representation in CBOR
Hashes	algorithm + lowercase hex/base64 value; SHA-256 baseline

Do not use floating-point values for identifiers, revisions, sequence numbers, DMX addresses, universe numbers, MIDI channel/note identifiers, or enum-like fields.

40. Implementation Phases for Grok

11. Protocol skeleton: repository/module layout, version constants, IDs, enums, envelope, error model, codec interfaces.
12. Schema + vectors: define JSON schemas and canonical JSON/CBOR vectors for core handshake, ack, state, heartbeat, config, and errors.
13. Swift/Python/Rust model parity: compile/import cleanly with no networking yet.
14. Codec parity: JSON + CBOR round-trip and cross-language vector tests.
15. Session engine: WebSocket transport, HELLO negotiation, sequencing, correlation, reconnect, bounded queues.
16. Discovery: multicast query/advertisement with jitter and deduplication.
17. Health: negotiated heartbeat, observer timeout/hysteresis, component health.
18. Profiles: Bridge first, then Prism/Conductor/Lyric semantic helpers.
19. CLI tools: acp-inspect and acp-sim in Python; optional Rust inspector for Bridge development.
20. Cross-language integration suite and fuzz/malformed-input suite.
21. Product adapters: integrate behind existing Prism control architecture and corresponding Conductor/Lyric/Bridge boundaries. Do not bypass product engines.

41. Acceptance Criteria

- Swift, Python, and Rust SDKs encode/decode the same golden vectors.
- A Swift peer and Rust peer can discover/connect, negotiate ACP 1.0, exchange heartbeat, issue a command, receive an ack, request a snapshot, disconnect, reconnect, and reconcile.
- Python simulator can impersonate Prism, Conductor, Lyric, or Bridge for testing.
- Unknown optional fields do not break v1 peers.
- Malformed input cannot crash the process.
- All outbound queues are bounded and their overflow/coalescing behavior is tested.
- Bridge blackout is idempotent, acknowledged, logged, and state-reflected.
- Config updates are validated and revision-safe.
- MIDI/device state is never marked confirmed solely because a command was transmitted.
- No ACP handler directly drives DMX inside Prism; it routes through the existing semantic control path.
- The full stack runs without Internet access.
- Wireshark Lua dissector decodes ACP discovery, CBOR WebSocket frames, and JSON diagnostic frames; required acp.* display filters work against supplied test captures.

42. Explicit Decisions Grok Must Not Re-Litigate

- Aurora is the ecosystem/family name; Prism is the lighting-control product.

- Swift, Python, and Rust implementations are mandatory.
- TOML remains the preferred human configuration format for Bridge, while ACP config messages remain typed.
- Bridge heartbeats and Conductor health/warning presentation are first-class requirements.
- MIDI transmission does not imply acknowledgement or state confirmation.
- Musical/show semantics are first-class. ACP is not merely a generic remote-button protocol.
- Prism's existing engine ownership and ControlActionRouter/behavior architecture remain authoritative.
- The protocol must support constrained Bridge builds without forcing the full desktop stack onto them.

43. Open Design Items Allowed During Implementation

These are implementation choices, not reasons to stall the protocol model work:

- Exact administratively scoped multicast group and UDP port.
- Exact WebSocket service port and optional mDNS service name if mDNS is added as a secondary discovery mechanism.
- Specific CBOR libraries per language.
- Exact TLS certificate provisioning UX for hardened deployments.
- Whether TypeScript JSON models are generated in v1 or deferred.
- Precise heartbeat defaults per product profile, provided they remain negotiated and bounded.

Any choice that changes wire semantics, message ownership, acknowledgement semantics, or compatibility rules must be documented as an ACP spec revision rather than hidden in one language implementation.

- Wireshark Lua dissector, coloring rules, installation instructions, filter cheat sheet, and sample PCAP/PCAPNG regression captures.

44. Definition of Done for the First Handoff

The first Grok delivery should not attempt to integrate every Aurora product. It should produce a clean protocol foundation that product work can consume.

- Language-neutral schemas and spec markdown committed.
- Swift, Python, Rust model packages with matching public concepts.
- CBOR and JSON codecs.
- WebSocket session abstraction with HELLO/ACK, sequencing, correlation, reconnect, heartbeat.
- UDP discovery implementation.
- Core health/state/config/command types.
- Bridge profile types including blackout and configuration.
- Skeleton Prism/Conductor/Lyric profile types.
- Golden vectors and cross-language tests.
- Python simulator/inspector sufficient to test a real Bridge or Prism adapter.
- Clear README showing how to add a new message type or capability without breaking compatibility.

Appendix A. Example End-to-End Flow

```
1. Bridge boots.
2. Bridge emits discovery.advertisement.
3. Conductor sees node_id + role=bridge + reliable endpoint.
4. Conductor opens WebSocket.
5. session.hello / session.hello_ack negotiate ACP 1.0 + CBOR.
6. Bridge begins health.heartbeat every negotiated interval.
```

```
7. Conductor requests state.snapshot for bridge.status + bridge.config revision.
8. Operator changes DMX universe in Conductor.
9. Conductor sends config.set(expected_revision=N).
10. Bridge validates, commits, returns config.result(new_revision=N+1).
11. Bridge emits state.delta for affected configuration/status.
12. Operator presses Bridge blackout in Conductor.
13. Conductor sends bridge.blackout(enabled=true, idempotency_key=...).
14. Bridge returns command.ack(applied).
15. Bridge emits authoritative state.delta blackout=true.
16. If Bridge disconnects, Conductor marks it offline only after timeout policy.
17. On reconnect, new instance_id reveals whether Bridge rebooted.
18. Conductor performs fresh handshake and snapshot reconciliation.
```

Appendix B. Example Prism GO Flow

```
Conductor
  → cue.go { target_context, idempotency_key }
Prism ACP adapter
  → validates permission + target
  → routes semantic action into Prism ControlActionRouter
Prism engine
  → advances authoritative cue state
  → resolves preview + physical output through existing engine path
Prism ACP adapter
  → command.ack { applied }
  → state.delta { current_cue, next_cue, revision }
Conductor
  → updates operator state from Prism's authoritative delta
```

Appendix C. Naming Conventions

Item	Convention	Example
Wire fields	snake_case	message_id
Message types	lowercase dotted	health.heartbeat
Capabilities	lowercase dotted	bridge.blackout
Error codes	lowercase dotted	config.revision_conflict
Swift	Swift idioms	messageID, sessionID
Python	snake_case	message_id
Rust	snake_case fields; idiomatic type names	message_id, MessageId

Appendix D. Handoff Note to Grok

BUILD ORDER Start with the protocol boundary and cross-language conformance, not product UI integration. A boring, deterministic protocol core will save enormous pain when Prism, Conductor, Lyric, and Bridge begin evolving in parallel.

When implementation details are ambiguous, prefer the choice that preserves: explicit ownership, deterministic behavior, bounded resource use, offline LAN operation, inspectability, and cross-language parity.

Asset Conformance, Show Manifests, and Lyric Assignment Profile

ACP v1.2 adds a formal asset-conformance layer so that a Conductor show selection identifies an exact, reproducible set of assets and revisions across Prism, Lyric, Remote, Bridge, and future Aurora products. Human-readable names are never sufficient to establish conformance.

Canonical Show Identity

A show is identified by a stable show_id plus an immutable revision and manifest hash. Conductor is authoritative for the statement 'this exact collection of revisions constitutes this show,' while each product remains authoritative for the assets and runtime state inside its own domain.

```
show_id: UUID
show_name: "Haywire Big Stage"
revision: 42
manifest_sha256: "..."
policy: "require_required_assets"
```

RULE A command such as 'load Haywire Big Stage' must resolve to an exact show_id, revision, and manifest hash before nodes are considered conformant.

Asset Identity

Field	Purpose
asset_id	Stable identity of the logical asset.
asset_type	Namespaced type such as prism.project, lyric.chart, bridge.config, remote.layout.
revision	Immutable monotonically advancing logical revision.
sha256	Content-integrity fingerprint for verification and transfer.
media_type	Optional MIME/media type for transferred resources.
size_bytes	Optional transfer/inventory metadata.
producer	Product/domain that owns or authored the asset.

Once an asset revision exists, its content must never be silently mutated. Any content change creates a new revision. This enables deterministic rollback, caching, comparison, and show snapshots.

Show Manifest

The show manifest lists the exact asset revisions expected by the production. It may include both shared assets and participant-specific requirements.

```
{
  "show_id": "...",
  "revision": 42,
  "assets": [
    {"asset_id":"...", "asset_type":"prism.project", "revision":19, "sha256":"...",
"requirement":"required"},
    {"asset_id":"...", "asset_type":"bridge.config", "revision":14, "sha256":"...",
"requirement":"required"},
    {"asset_id":"...", "asset_type":"remote.layout", "revision":6, "sha256":"...",
"requirement":"optional"}
  ],
  "participant_assignments": [ ... ]
}
```

Requirement Policy

Requirement	Meaning

required	Mismatch prevents normal ARM unless operator explicitly overrides.
recommended	Mismatch produces a visible warning but may allow ARM.
optional	Absence does not block show readiness.

Conductor must provide a deliberate Force Arm/Override mechanism for nonconformant situations. Overrides are logged with the exact deviations present at arm time.

Show Preparation Lifecycle

22. SELECT - Conductor selects a stable show_id and exact revision.
23. PREPARE - Participants load local assets and return inventory/conformance information.
24. VERIFY - Conductor compares participant state against the manifest and initiates synchronization where allowed.
25. ARM - Required participants acknowledge the exact manifest revision/hash they are prepared to use.
26. LIVE - Conductor begins live show operation with the armed manifest as the session baseline.

ACP must support show.prepare, show.conformance, show.verify, show.arm, show.armed, and show.disarm semantics. Reconnection during LIVE triggers reconciliation against the armed manifest rather than assuming the returning node is still correct.

Conformance States

State	Meaning
ready	All required assigned assets are present and verified.
syncing	Synchronization is in progress.
missing	A required assigned asset is absent.
stale	Correct logical asset exists but at an older revision.
hash_mismatch	Asset identity/revision claims match but content fingerprint does not.
incompatible	Asset cannot be used by this product/client capability set.
degraded	Can participate with known nonfatal deviations.
failed	Preparation or validation failed.

Resource Synchronization

ACP resource.transfer is the mechanism for correcting asset mismatches. Ordinary control envelopes must not carry large project/chart files.

- Sender offers asset metadata including asset_id, revision, size, media type, and SHA-256.
- Receiver accepts or rejects the transfer based on capacity, policy, and compatibility.
- Transfer may use chunked ACP messages for modest resources or a negotiated authenticated local HTTP endpoint for larger resources.
- Receiver stages the asset, verifies SHA-256, validates readability/compatibility, and only then atomically activates it.
- Transfer completion and asset activation are separate acknowledgements.
- A failed update must leave the previously valid asset intact whenever possible.

Lyric: Song, Chart, Participant, and Device Are Separate Identities

Lyric must support different charts for different performers while all performers remain synchronized to the same canonical song and show position. ACP therefore treats song identity, chart identity, participant identity, and device identity as independent concepts.

Identity	Question answered
song_id	What musical piece is being performed?
chart_id / asset_id	Which specific representation of that song is this?
participant_id	Which performer is this presentation intended for?
device_id / node_id	Which Lyric device is currently presenting it?

CRITICAL LYRIC RULE A Lyric client must never infer its chart solely from song_id. Conductor resolves and transmits the participant's exact chart assignment.

Chart Metadata

```
{
  "asset_id": "...",
  "asset_type": "lyric.chart",
  "song_id": "...",
  "revision": 11,
  "sha256": "...",
  "chart_type": "chord_lyrics",
  "instrument": "guitar",
  "variant": "full_band",
  "transposition": 0,
  "display_metadata": {
    "preferred_orientation": "portrait",
    "page_count": 3,
    "supports_auto_scroll": true
  }
}
```

chart_type is extensible. Initial well-known values should include nashville_number, chord_lyrics, lyrics_only, lead_sheet, sheet_music, and custom. Instrument and variant are optional metadata and must not be used as identity.

Participant Preferences and Explicit Assignments

Conductor owns chart-assignment management. A participant may have a persistent preference such as Nashville Number charts, but a specific song may override that preference.

```
participant.preference
{
  "participant_id": "...",
  "preferred_chart_type": "nashville_number"
}

chart.assignment
{
  "show_id": "...",
  "song_id": "...",
  "participant_id": "...",
  "device_id": "...",
  "chart": {
    "asset_id": "...",
    "revision": 8,
    "sha256": "..."
  },
  "reason": "performer_preference"
}
```

Assignment Resolution

Assignment resolution occurs in Conductor, not independently in Lyric. Recommended precedence:

27. Explicit song + participant assignment.
28. Participant chart preference.

29. Show/band default chart policy.
30. Compatible fallback chart, if fallback is permitted.

The resolved assignment is transmitted over ACP. The optional reason field should use stable values such as explicit_assignment, performer_preference, show_default, or fallback so diagnostics can explain why a chart was selected.

Participant Binding

Preferences belong primarily to performers, not physical tablets. A Lyric device binds to a participant for a session. Replacing or reassigning an iPad must not require recreating the performer's chart preferences.

```
lyric.client_ready
{
  "node_id": "...",
  "device_id": "...",
  "participant_id": "...",
  "supported_chart_types": [
    "nashville_number",
    "chord_lyrics",
    "lyrics_only"
  ]
}
```

Conductor may change participant binding explicitly. Any binding change causes chart assignments and conformance to be recalculated before the client is considered ready.

Per-Participant Conformance

Lyric conformance is evaluated against the assets assigned to that participant, not against every chart available in the show. A client may optionally cache the full chart library for redundancy, but that cache is not a readiness requirement unless policy says otherwise.

```
lyric.assignment_status
{
  "participant_id": "...",
  "show_id": "...",
  "show_revision": 42,
  "assigned_count": 42,
  "verified_count": 41,
  "status": "missing",
  "problems": [
    {
      "song_id": "...",
      "asset_id": "...",
      "required_revision": 3,
      "state": "missing"
    }
  ]
}
```

Conductor Readiness Matrix

ACP must expose enough structured information for Conductor to display readiness at both product and participant level. A single generic 'Lyric ready' flag is insufficient.

```
HAYWIRE BIG STAGE - REV 42

Prism Main             READY
Bridge Stage Left      READY
Bridge Stage Right     STALE CONFIG
Remote FOH             READY

Lyric / Dakota         READY - 42/42 assigned charts verified
```

```
Lyric / Sean           READY - 42/42 assigned charts verified
Lyric / Rick           MISSING - 41/42 assigned charts verified
Lyric / Clint          READY - 42/42 assigned charts verified
```

Required Asset and Assignment Message Families

Message family	Purpose
manifest.get / manifest.report	Retrieve or report the exact show manifest.
asset.inventory / asset.status	Report locally available assets and revisions.
asset.required	Declare a required/recommended/optional asset.
asset.sync / asset.sync_result	Request and report synchronization.
resource.offer / accept / complete / activate	Transfer and atomically activate content.
participant.identity / participant.assignment	Identify performers and session bindings.
chart.catalog / chart.metadata	Describe available chart representations.
chart.assignment / chart.assignment_set	Transmit resolved participant-specific chart choices.
lyric.assignment_manifest	Declare all chart assets assigned to one participant/client.
lyric.assignment_status / lyric.ready	Report per-participant conformance and readiness.

Wireshark Requirements for Asset Conformance

The ACP Wireshark dissector added in v1.1 must expose asset/show/assignment fields so synchronization failures can be diagnosed directly from captures.

- acp.show.id, acp.show.revision, acp.show.manifest_sha256
- acp.asset.id, acp.asset.type, acp.asset.revision, acp.asset.sha256
- acp.asset.status, acp.asset.requirement
- acp.participant.id, acp.device.id
- acp.chart.type, acp.chart.assignment_reason
- acp.resource.transfer_id, acp.resource.status

```
Example display filters:

acp.type == "show.conformance" && acp.asset.status != "ready"

acp.participant.id == "<participant-id>" && acp.asset.status == "missing"

acp.type == "chart.assignment"

acp.asset.status == "hash_mismatch"

acp.show.revision == 42 && acp.type == "show.armed"
```

Asset-Conformance Acceptance Criteria

- Conductor can prepare an exact show revision and receive structured conformance from Prism, Lyric, Remote, Bridge, and simulated future products.
- A same-name but different-revision show is never considered conformant.
- A hash mismatch is distinguishable from a stale revision.
- Required, recommended, and optional dependencies produce different readiness behavior.
- A stale Bridge configuration can be synchronized, validated, atomically activated, and reported ready.
- Different Lyric participants can be assigned different chart assets for the same song.
- Changing a Lyric device does not destroy participant chart preferences.
- A participant-specific explicit chart assignment overrides the participant default preference.
- Lyric readiness is based on that participant's resolved assignments.
- Resource transfer verifies SHA-256 before activation and preserves the prior valid asset on failed activation.

- Force Arm records all known deviations.
- Wireshark can filter show, asset, participant, chart-assignment, and synchronization traffic using stable acp.* fields.

Implementation Note to Grok - ACP v1.2

DO THIS BEFORE FREEZING THE SDK MODELS AssetRef, ShowManifest, AssetRequirement, AssetConformance, ParticipantIdentity, ParticipantBinding, ChartMetadata, ChartAssignment, AssignmentManifest, ResourceTransfer metadata, and show preparation lifecycle types must exist in the language-neutral schema and in Swift, Python, and Rust before product integration begins.